Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 06

Motor de plantillas con Django

DOCENTE:

Michael Montgomery Rosell

CURSO:

Desarrollo de Aplicaciones Empresariales

Desarrollo de Aplicaciones Web:
Motor de plantillas con Django

I. Elemento de la capacidad terminal de la semana

- Implementar un motor de plantillas en una aplicación Web en Django.

II. Fundamento teórico

- Revisar el material del campus virtual correspondiente a la semana y los fundamentos teóricos del tema antes del desarrollo del laboratorio.
- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

III. Normas empleadas

- No aplica

IV. Recursos

- En este laboratorio cada alumno trabajará con un equipo con Windows 10 o Windows 11.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual.

VII.- PROCEDIMIENTO:

Actividad 1: Revisión de conceptos / Marcos de trabajo

Relacione cada concepto con su definición. Complete la tabla con sus propias palabras.

Concepto / Principio	Definición, comentarios o interpretación
1. Sistema de templates de Django (DTL)	Es el motor de plantillas integrado en Django (Django Template Language), este permite generar páginas HTML dinámicas combinando estructuras basadas en etiquetas HTML con información proveniente de las vistas y los modelos de la aplicación. Esta herramienta facilita la separación entre la lógica de la aplicación y la presentación de los contenidos.
2. Herencia de plantillas (extends / block)	Es un mecanismo que permite reutilizar estructuras HTML comunes entre diferentes vistas y presentaciones, evitando repetir código y facilitando el mantenimiento de la aplicación. Mediante {% extends %} una plantilla hereda la estructura de otra, mientras que {% block %} define secciones que pueden personalizarse o sobrescribirse.
3. Tags de template ({% %}) y variables ({{ }})	Son elementos del lenguaje de plantillas integrado en Django los cuales permiten incorporar contenido dinámico en HTML. Los tags {% %} ejecutan instrucciones o estructuras de control, como condicionales, ciclos y herencia; mientras que las variables {{ }} muestran información proporcionada por las vistas, como nombres, precios, categorías, etc.
4. Archivos estáticos ({% static %})	Se tratan de recursos que no cambian dinámicamente según las solicitudes del usuario, como archivos CSS, JavaScript, imágenes e iconos. Django nos permite gestionar sus rutas mediante el tag {% static %}, el cual genera la dirección correspondiente para incluirlos correctamente en las plantillas HTML.

Actividad 2: Desarrollo práctico / Ejecución

Siga los pasos detallados a continuación para completar el procedimiento del laboratorio.

Creación del entorno Virtual

1. Crea una carpeta en la unidad "**C:**" llamada "**djangoApp06**".
2. Este comando creará un directorio llamado "**myvenv**" que contiene nuestro entorno virtual:

```
C:\djangoApp06> python -m venv venv
```

3. Iniciar el entorno virtual:

```
C:\djangoApp06> venv\Scripts\activate
```

Proyecto en Django

4. En la consola, ejecuta

```
(venv)C:\djangoApp06> pip install django
```

5. Para crear el proyecto, desde tu entorno virtual ejecuta la siguiente línea de comando:

```
(venv)C:\djangoApp06> django-admin startproject lab06
```

Figura 1. Creación del Proyecto "lab06"

```
(venv) C:\Users\USUARIO\Documents\Proyectos\Python\DAE>
django-admin startproject lab06
```

6. Ingrese a la carpeta de "lab06"

```
(venv)C:\djangoApp06> cd lab06
```

Figura 2. Ingreso al Directorio \lab06

```
(venv) C:\Users\USUARIO\Documents\Proyectos\Python\DAE>
cd lab06

(venv) C:\Users\USUARIO\Documents\Proyectos\Python\DAE\
lab06>
```

Creación de una aplicación de "tienda"

1. Ahora que su entorno "**lab06**" está configurado, está listo para comenzar a trabajar. Para crear su aplicación, asegúrese de estar en el mismo directorio que manage.py y escriba este comando:

```
(venv)C:\djangoApp06\lab06> python manage.py startapp tienda
```

Figura 3. Creación de la Aplicación "tienda"

```
(venv) C:\Users\USUARIO\Documents\Proyectos\Python\DAE\lab06>
python manage.py startapp tienda
```

2. Edita "INSTALED_APPS" del archivo "**lab06\settings.py**" de la siguiente manera:

```
INSTALLED_APPS = [
    'tienda.apps.TiendaConfig',
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
```

Creación de la base de datos con SQLite3

3. Crear las tablas en la base de datos, ejecute el siguiente comando:

```
(venv)C:\djangoApp06\lab06> python manage.py migrate
```

Figura 4. Creación de la base de datos

```
(venv) C:\Users\USUARIO\Documents\Proyectos\Python\DAE\lab06>
python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... O
K
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
```

Creación del usuario "admin"

4. Dentro del entorno virtual de tu proyecto, ejecuta el siguiente código para crear al usuario administrador

```
(venv)C:\djangoApp06\lab06> python manage.py createsuperuser
```

Figura 5. Creación del usuario "admin"

```
(venv) C:\Users\USUARIO\Documents\Proyectos\Python\DAE\lab06
python manage.py createsuperuser
Username (leave blank to use 'usuario'): admin
Email address: cesar.bustos@tecsup.edu.pe
Password:
Password (again):
The password is too similar to the email address.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

5. Registrar como usuario "admin" y como contraseña "Tecsup2023"
6. Levanta el servidor Web

```
(venv)C:\djangoApp06\lab06> python manage.py runserver
```

Figura 6. Inicialización del servidor

```
(venv) C:\Users\USUARIO\Documents\Proyectos\Python\DAE\lab06>
python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
November 07, 2026 - 01:55:52
Django version 6.1.2, using settings 'lab06.settings'
Starting WSGI development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a pro
duction setting. Use a production WSGI or ASGI server instead
.
For more information on production servers see: https://docs.
djangoproject.com/en/6.1/howto/deployment/
```

7. Visita la siguiente url e ingresa con las credenciales creadas anteriormente.

```
http://127.0.0.1:8000/admin
```

Figura 7. Inicio de sesión como "admin"

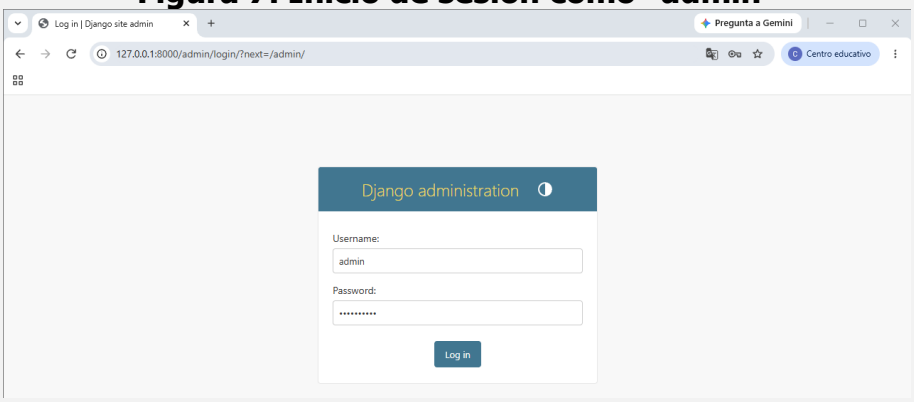
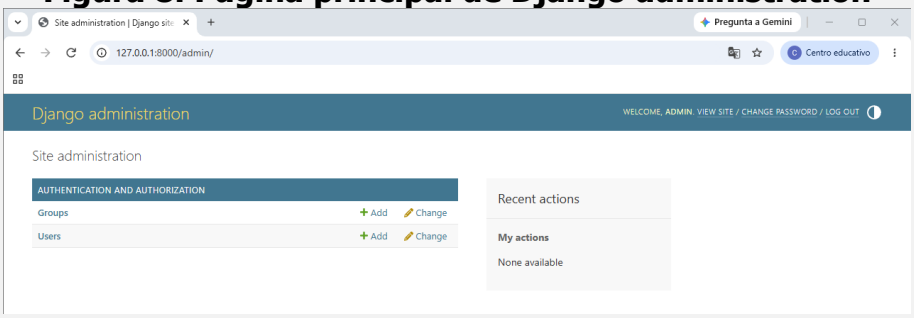


Figura 8. Página principal de Django administration



Creación de los modelos

8. Crear una clase llamada **"Categoria"**. Editar el archivo **"tienda/models.py"** de la siguiente manera:

```
class Categoria(models.Model):
    nombre = models.CharField(max_length=200)
    pub_date = models.DateTimeField('fecha de registro',auto_now=True)

    def __str__(self):
        return self.nombre
```

9. Crear otras clase llamada **"Producto"** con el siguiente código:

```
class Producto(models.Model):
    categoria = models.ForeignKey(Categoria,on_delete=models.CASCADE)
    nombre = models.CharField(max_length=200)
    precio = models.DecimalField(max_digits=6,decimal_places=2)
    stock = models.IntegerField(default=0)
    pub_date = models.DateTimeField('date published')

    def __str__(self):
        return self.nombre
```

10. Incluir la aplicación "tienda" para crear la base de datos, ejecutar el siguiente comando:

```
(venv)C:\djangoApp06\lab06> python manage.py makemigrations
```

Figura 9. Creación de migraciones para Categoría y Producto

```
(venv) C:\Users\USUARIO\Documents\Proyectos\Python\DAE\lab06
python manage.py makemigrations
Migrations for 'tienda':
  tienda\migrations\0001_initial.py
    + Create model Categoria
    + Create model Producto
```

```
Migrations for 'tienda':
  tienda\migrations\0001_initial.py
    - Create model Categoria
    - Create model Producto
```

11. Ejecutar de nuevo el comando **"migrate"** para crear esas tablas modelos en su base de datos:

```
(venv)C:\djangoApp06\lab06> python manage.py migrate
```

Figura 10. Gestión de migraciones para Categoría y Producto

```
(venv) C:\Users\USUARIO\Documents\Proyectos\Python\DAE\lab06
python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions,
 tienda
Running migrations:
  Applying tienda.0001_initial... OK
```

Registro de nuevos módulos al administrador

12. En el archivo **"lab06/tienda/admin.py"**, registra la clase **"Categoria"** y **"Producto"** para poder administrarlo desde el Admin de Django.

```python
from django.contrib import admin

# Register your models here.
from .models import Categoria
from .models import Producto

admin.site.register(Categoria)
admin.site.register(Producto)
```

13. Refresca la página del administrador y verifica que puedas ver los módulos.
14. Añade los siguientes productos y categorías respectivamente:

Categoría	Producto	Precio	Stock
Gaseosas	Fanta 500ml	1.80	10
Gaseosas	Coca Cola 500ml	2.50	10
Gaseosas	7up 500ml	1.70	10
Snacks	Cheetos	1.00	12
Snacks	Chizito	1.00	12
Galletas	Picaras	1.00	12
Galletas	Coronita Fresa	0.70	12
Chocolates	Princesa	1.50	8
Chocolates	Sublime	1.50	8
Chocolates	Triángulo	1.20	8

Figura 11. Registro de Categorías

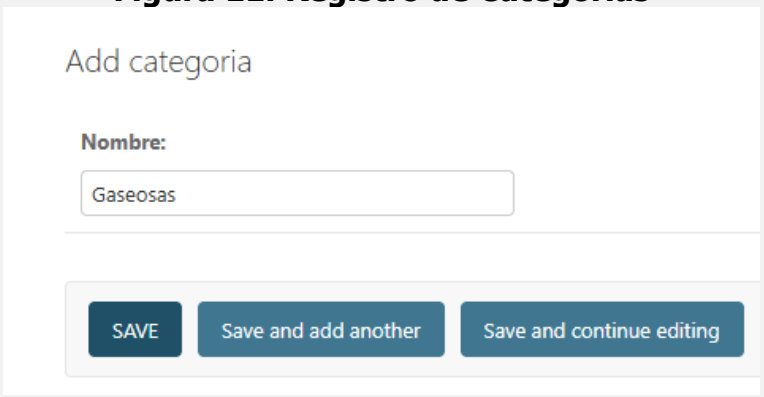


Figura 12. Tabla "Categorías"

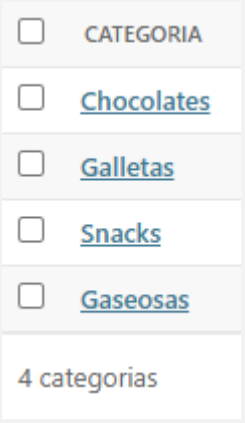


Figura 13. Registro de Productos

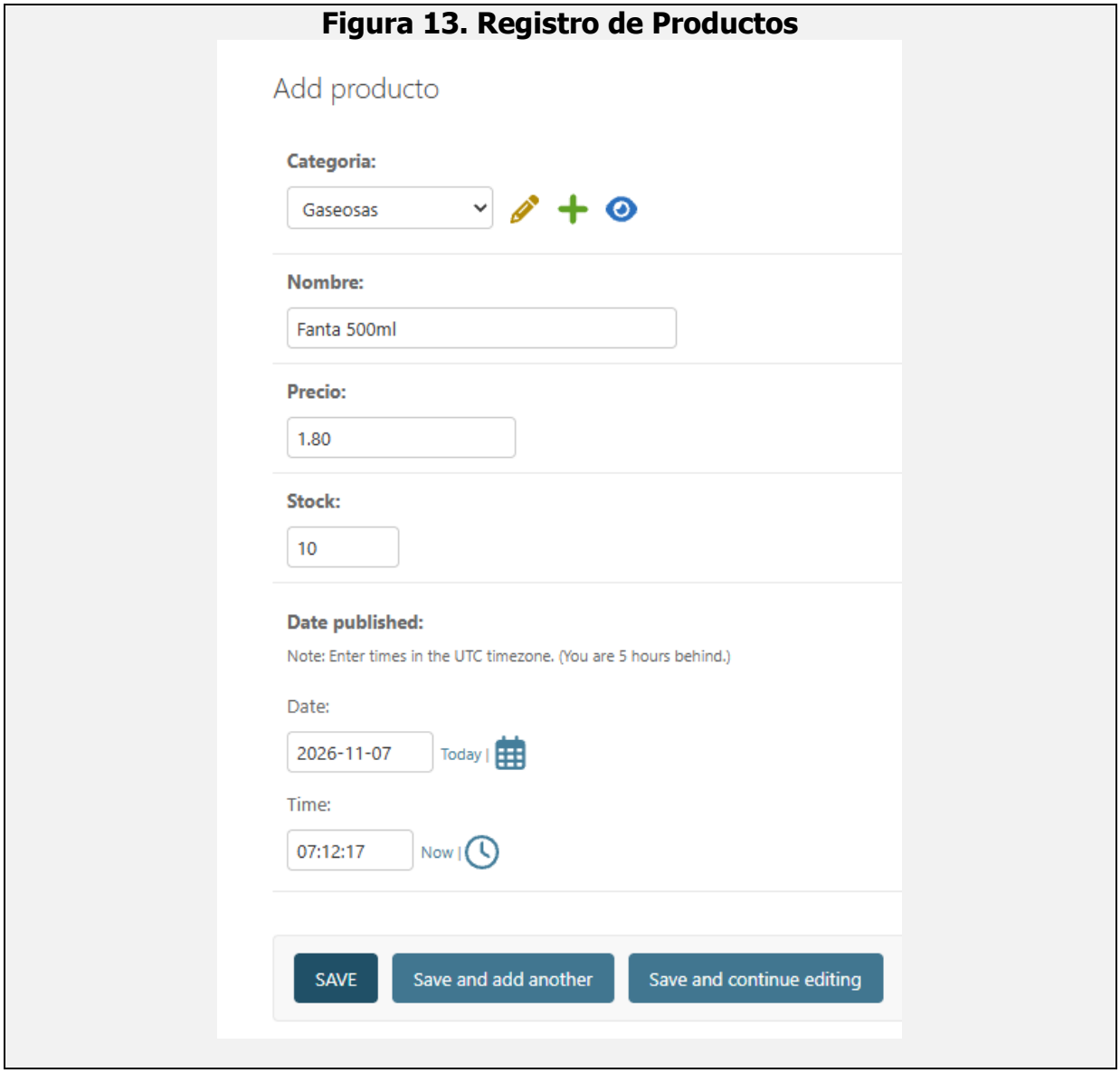
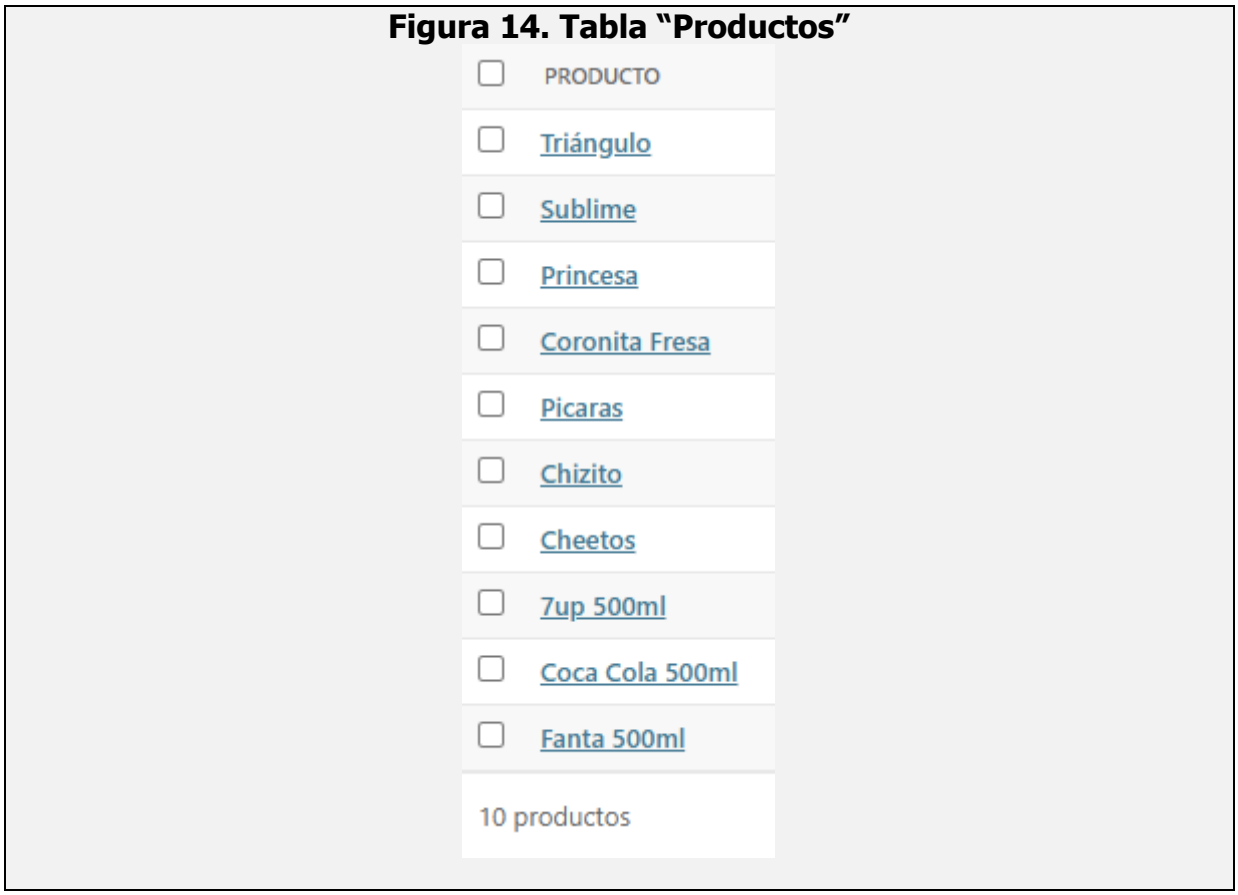


Figura 14. Tabla "Productos"



Creación de las vistas

15. Dentro de la carpeta **"lab06/tienda"**, crear la carpeta **"templates"**.
16. Dentro de la carpeta **"templates"** crea el archivo index.html con el siguiente contenido:
    <h1>Inicio</h1>
17. Dentro de la carpeta **"templates"** crea el archivo producto.html con el siguiente contenido:
    <h1>Producto</h1>.
18. Dentro de la carpeta lab06/tienda ingresar el siguiente código en el archivo views.py

```
from django.shortcuts import render

# Create your views here.
def index(request):
    return render(request,'index.html')

def producto (request):
    return render(request,'producto.html')
```

Rutas de la aplicación

19. Dentro del directorio **"lab06/tienda"**, crear un archivo llamado **"urls.py"**

20. En el archivo **"tienda/urls.py"** incluya el siguiente código:

```
from django.urls import path

from . import views

app_name = 'tienda'

urlpatterns = [
    path('', views.index, name='index'),
    path('producto', views.producto, name='producto')
]
```

21. El siguiente paso es apuntar al URLconf raíz en el módulo tienda.urls. En **"lab06/urls.py"**, agregue una importación para django.urls.include e inserte un include () en la lista urlpatterns, de la siguiente manera:

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('tienda/', include('tienda.urls')),
    path('admin/', admin.site.urls),
]
```

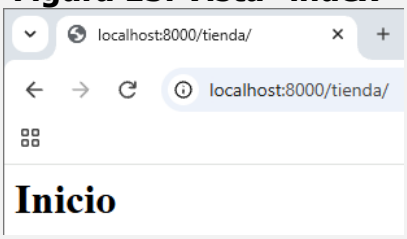
22. Ejecute el siguiente comando:

```
(venv)C:\djangoApp06\lab06> python manage.py runserver
```

23. Visite la siguiente URL desde su navegador web:

```
http://localhost:8000/tienda
```

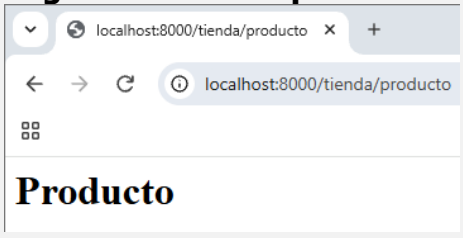
Figura 15. Vista "index"



24. Visite la siguiente URL desde su navegador web:

```
http://localhost:8000/tienda/producto
```

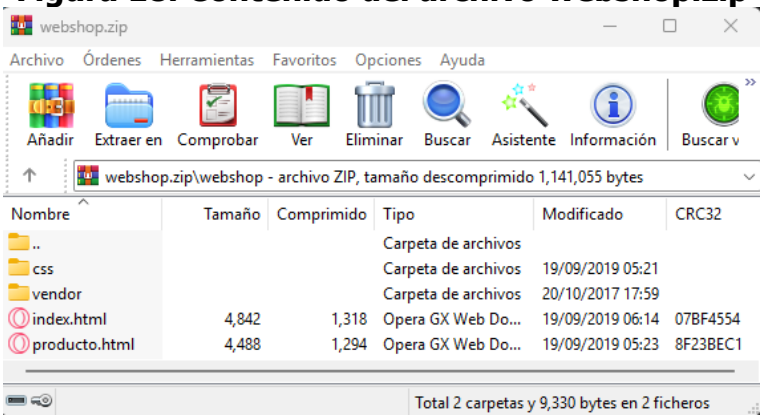
Figura 16. Vista "producto"



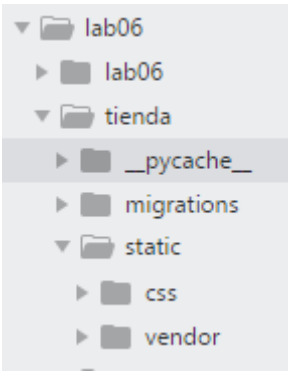
Publicación de los archivos estáticos (css, js, etc)

25. Solicita al instructor los el archivo webshop.zip, y descomprime el archivo.

Figura 18. Contenido del archivo webshop.zip



26. Dentro de la carpeta **"lab06/tienda"**, crear la carpeta **"static"**.
27. Dentro de la carpeta **"static"**, copia las carpetas **"css"** y **"vendor"** para obtender la siguiente estructura:



Desarrollo de la plantilla

28. Dentro de la carpeta **"templates"** crea el archivo **"layout.html"** con el contenido del archivo **"webshop/index.html"**.
29. Añade en la primera línea "**{% load static%}**" para cargar los archivos estáticos. Luego modifica las rutas de las hojas de estilos con el siguiente código: "**{% static 'ruta/del/archivo' %}**" para cargar las hojas de estilos correctamente.
30.

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1,
    <title>Shop Homepage</title>
    <link href="{% static 'vendor/bootstrap/css/bootstrap.min.css' %}" rel="stylesheet">
    <link href="{% static 'css/shop.css' %}" rel="stylesheet">
  </head>
```

Utiliza el código anterior para cargar correctamente los archivos **"js"** que se encuentran al final del archivo.

```
<!-- Bootstrap core JavaScript -->
<script src="{% static 'vendor/jquery/jquery.min.js' %}"></script>
<script src="{% static 'vendor/bootstrap/js/bootstrap.bundle.min.js' %}"><
```

31. Ubícate en la línea 59 aproximadamente de tu archivo **"templates/layout.html"**, corta todo el contenido que hay dentro de los comentarios **"Block Content"**:

```
58          <!-- Block Content -->
59          <div id="carouselExampleIndicators" 
84          </div>
85
86          <div class="row">...
102         </div>
103         <!-- /Block Content -->
```

Y pégalo en el archivo **"templates/index.html"** de la siguiente manera:

```
1   {% extends "layout.html" %}
2   {% block content %}
3   <div id="carouselExampleIndicators" class="carousel slide
28  </div>
29
30  <div class="row">...
46  </div>
47  {% endblock %}
```

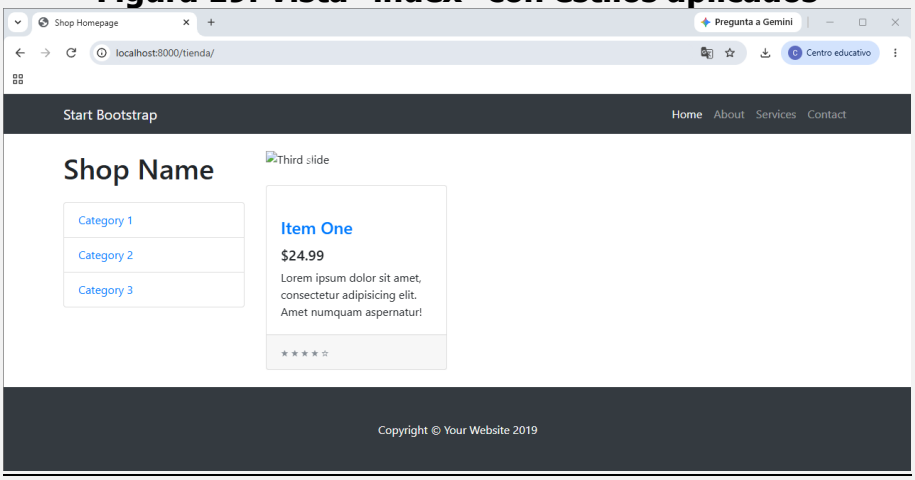
32. Finalmente, modifica el espacio cortado de tu archivo **"templates/layout.html"**, de la siguiente manera indicando un nuevo bloque.

```
<div class="col-lg-9">
    <!-- Block Content -->
    {% block content %}{% endblock %}
    <!-- /.BlockContent -->
</div>
<!-- /.col-lg-9 -->
```

33. Visite la siguiente URL desde su navegador web y verifique que se visualice correctamente.

```
http://localhost:8080/tienda
```

Figura 19. Vista "index" con estilos aplicados



Cargando datos en la vista principal

34. A continuación, vamos a editar el archivo **"tienda/views.py"** de la siguiente manera para la vista principal (index):

```
From .models import Producto
def index(request):
    product_list = Producto.objects.order_by('nombre')[:6]
    context = {'product_list': product_list}
    return render(request, 'index.html', context)
```

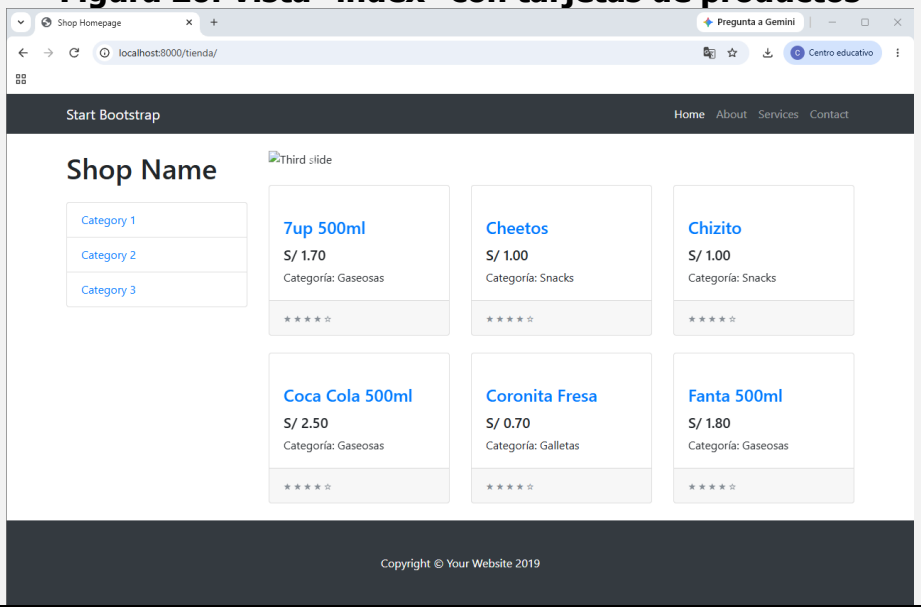
35. Modificar en la vista index.html, el listado de los productos, de la siguiente manera:

```
<div class="row">
    {% for prod in product_list %}
    <div class="col-lg-4 col-md-6 mb-4">
      <div class="card h-100">
        <a href="producto/{{ prod.id }}/">
            <img class="card-img-top" src="http://placehold.it/700x400" al
        </a>
        <div class="card-body">
          <h4 class="card-title">
            <a href="producto/{{ prod.id }}/">{{ prod.nombre }}</a>
          </h4>
          <h5>S/ {{ prod.precio }}</h5>
          <p class="card-text">Categoría: {{ prod.categoria.nombre }}</p>
        </div>
        <div class="card-footer">
          <small class="text-muted">&#9733; &#9733; &#9733; &#9733; &#9734
        </div>
      </div>
    </div>
    {% endfor %}
</div>
```

36. Visite la siguiente URL desde su navegador web.

http://localhost:8080/tienda

Figura 20. Vista "index" con tarjetas de productos



Cargando datos en la vista "producto"

37. Modifica el archivo **"tienda/urls.py"** de la siguiente manera para que la vista producto pueda recibir el id respectivo:

```
urlpatterns = [
    path('', views.index, name='index'),
    path('producto/<int:producto_id>/', views.producto, name='producto')
]
```

38. A continuación, vamos a editar el archivo **"tienda/views.py"** de la siguiente manera para que pueda capturar el id enviado desde la url:

```
from django.shortcuts import get_object_or_404, render

from .models import Producto

# Create your views here.
def index(request):
    product_list = Producto.objects.order_by('nombre')[:6]
    context = {'product_list': product_list}
    return render(request, 'index.html', context)

def producto(request, producto_id):
    producto = get_object_or_404(Producto, pk=producto_id)
    return render(request, 'producto.html', {'producto': producto})
```

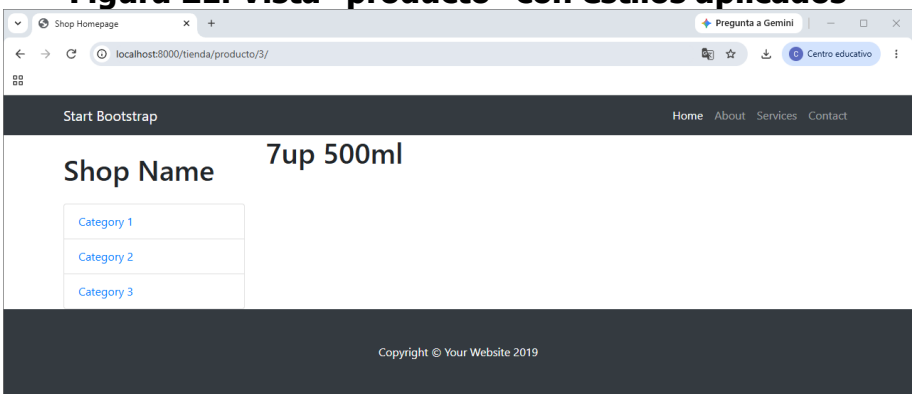
39. Modifica el archivo **"templates/producto.html"** de la siguiente manera:

```
{% extends "layout.html" %}
{% block content %}
<h1>{{ producto.nombre }}</h1>
{% endblock %}
```

40. Visite la siguiente URL desde su navegador web y prueba el correcto funcionamiento:

http://localhost:8080/tienda

Figura 21. Vista "producto" con estilos aplicados



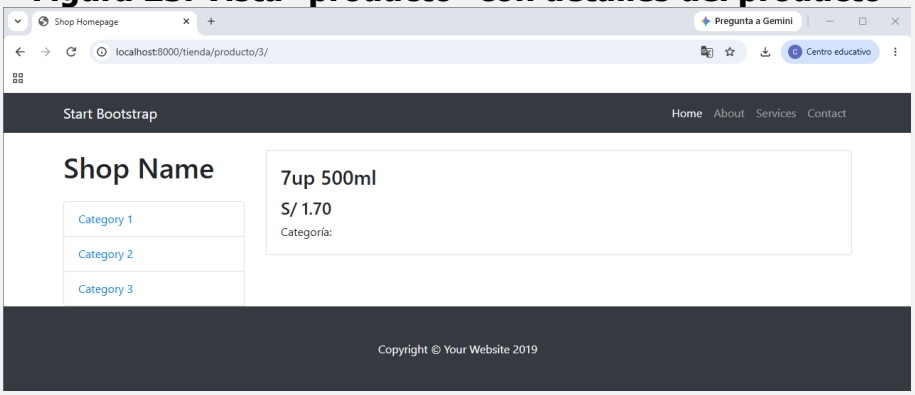
41. Aplica el diseño correspondiente a la vista del detalle del producto.

Figura 22. Código aplicado para "producto.html"

tienda > templates > producto.html > ...

```
{% extends "layout.html" %}
{% block content %}
<div class="card mt-4">
    <img class="card-img-top img-fluid" src="http://placehold.it/900x400" alt="">
    <div class="card-body">
        <h3 class="card-title">{{ producto.nombre }}</h3>
        <h4>S/ {{ producto.precio }}</h4>
        <p class="card-text">Categoría: {{ prod.categoria.nombre }}</p>
    </div>
</div>
{% endblock %}
```

Figura 23. Vista "producto" con detalles del producto



En caso no carguen los estilos en **settings.py** agregar debajo de **STATIC_URL = 'static/'**
STATICFILES_DIRS = [BASE_DIR / 'static/']

Actividad 3: Aplicación / Caso de estudio (Proyecto integrador)

Aplicando lo aprendido en las actividades anteriores, desarrolle el siguiente caso práctico.

- **Descripción del caso:** Utilizando los conocimientos de motor de plantillas con django, desarrolle una funcionalidad adicional que extienda el trabajo realizado en la actividad práctica. (Vea el detalle en la seccion TAREA)
- **Entregable esperado:** Capturas de pantalla del código implementado, del resultado de ejecución y una grabación de video (máximo 4 minutos) explicando el código y su funcionamiento.

TAREA

- Desarrolla el código correspondiente para cargar las categorías en el menú lateral, tanto en la página de inicio como en el detalle.
- Crea una nueva vista donde se liste todos los productos de una categoría al hacer clic en una opción del menú lateral de categorías.
- Agregar una foto a cada categoría y producto

Figura 24. Instalación de librería Pillow en Python para agregar imágenes a Categorías y Productos

```
(venv) C:\Users\Pc\Documents\TECSUP\CICLO IV\DesAplicEmpresariales\Lab6\lab06>pip install pillow
Collecting pillow
  Downloading pillow-13.0.0-cp314-cp314-win_amd64.whl.metadata (9.3 kB)
Downloading pillow-13.0.0-cp314-cp314-win_amd64.whl (7.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.3/7.3 MB 3.2 MB/s  0:00:02
Installing collected packages: pillow
Successfully installed pillow-13.0.0

[notice] A new release of pip is available: 26.2.1 -> 26.3
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Figura 25. Ruta para guardar imágenes

```
settings.py ×   urls.py 5   models.py 2
lab06 > settings.py > ...
130
131   # Configuración de archivos media (imgs de la app)
132   MEDIA_URL = '/media/'
133   MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
134
```

Figura 26. Modificación a urls.py para permitir la descarga/visualización de archivos

urls.py 5 models.py 2

lab06 > urls.py > ...

```python
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('tienda/', include('tienda.urls')),
    path('admin/', admin.site.urls),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)

```

Figura 27. Agregamos los campos de imagen a Categoria y Producto

models.py 2

tienda > models.py > Producto > imagen

```python
from django.db import models

# Create your models here.
class Categoria(models.Model):
    nombre = models.CharField(max_length=200)
    imagen = models.ImageField(upload_to='categorias/', null=True, blank=True)
    pub_date = models.DateTimeField('fecha de registro',auto_now=True)

    def __str__(self):
        return self.nombre

class Producto(models.Model):
    categoria = models.ForeignKey(Categoria,on_delete=models.CASCADE)
    nombre = models.CharField(max_length=200)
    precio = models.DecimalField(max_digits=6,decimal_places=2)
    stock = models.IntegerField(default=0)
    imagen = models.ImageField(upload_to='productos/', null=True, blank=True)
    pub_date = models.DateTimeField('date published')

    def __str__(self):
        return self.nombre
```

Figura 28. Realizamos las migraciones correspondientes

```
(venv) C:\Users\Pc\Documents\TECSUP\CICLO IV\DesAplicEmpresariales\Lab6\lab06>python manage.py makemigrations
Migrations for 'tienda':
  tienda\migrations\0002_categoria_imagen_producto_imagen.py
    + Add field imagen to categoria
    + Add field imagen to producto

(venv) C:\Users\Pc\Documents\TECSUP\CICLO IV\DesAplicEmpresariales\Lab6\lab06>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, tienda
Running migrations:
  Applying tienda.0002_categoria_imagen_producto_imagen... OK
```

Figura 29. Archivo context_processors.py para que las categorías carguen dinámicamente en el menú lateral tanto en inicio, detalle y filtrado

context_processors.py settings.py

tienda > context_processors.py > ...

```python
from .models import Categoria

def categorias_processor(request):
    return {
        'categorias_list': Categoria.objects.all()
    }
```

Figura 30. Registro del Context Processor en settings.py

context_processors.py settings.py

lab06 > settings.py > ...

```python
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
                'tienda.context_processors.categorias_processor',
```

Figura 31. Actualizamos tienda/views.py para incluir la vista productos_por_categoria

views.py

tienda > views.py > ...

```python
def productos_por_categoria(request, categoria_id):
    categoria_seleccionada = get_object_or_404(Categoria, pk=categoria_id)
    product_list = Producto.objects.filter(categoria=categoria_seleccionada)
    context = {
        'categoria_seleccionada': categoria_seleccionada,
        'product_list': product_list
    }
    return render(request, 'index.html', context)
```

Figura 32. Actualizamos tienda/urls.py para registrar la ruta de filtrado de productos por categoría

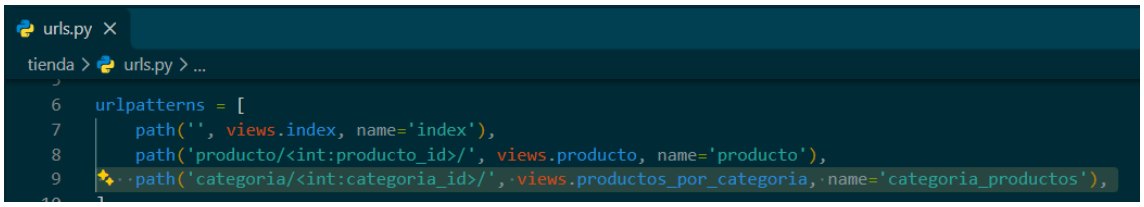


Figura 33. Modificación de Plantillas (Templates) para el menú lateral

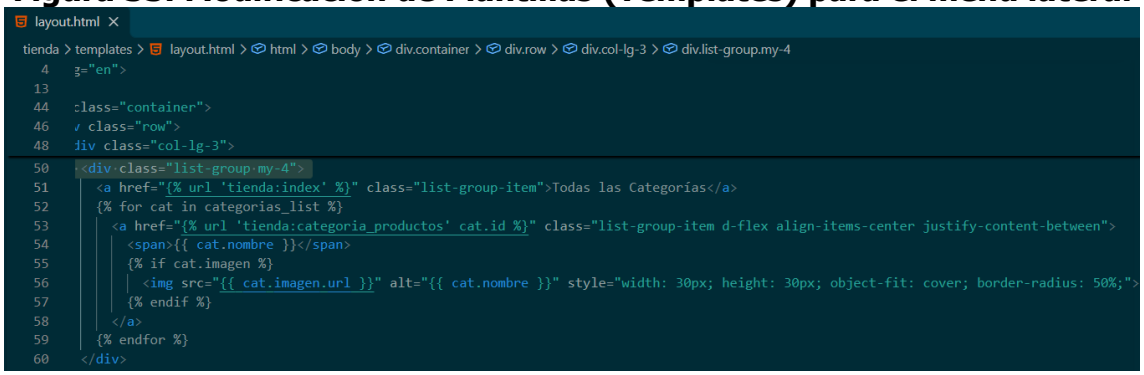


Figura 34. Actualización del renderizado de productos para incluir la imagen real del producto y el título de la categoría filtrada

index.html

tienda > templates > index.html > ...

```
{% extends "layout.html" %}

{% block content %}
{% if categoria_seleccionada %}
  <h2 class="my-4">Productos en "{{ categoria_seleccionada.nombre }}"</h2>
{% else %}
  <div id="carouselExampleIndicators" class="carousel slide my-4" data-ride="carousel">
    <ol class="carousel-indicators">
      <li data-target="#carouselExampleIndicators" data-slide-to="0" class="active"></li>
      <li data-target="#carouselExampleIndicators" data-slide-to="1"></li>
      <li data-target="#carouselExampleIndicators" data-slide-to="2"></li>
    </ol>
    <div class="carousel-inner" role="listbox">
      <div class="carousel-item active">
        <img class="d-block img-fluid" src="http://placehold.it/900x350" alt="First slide">
      </div>
      <div class="carousel-item">
        <img class="d-block img-fluid" src="http://placehold.it/900x350" alt="Second slide">
      </div>
      <div class="carousel-item">
        <img class="d-block img-fluid" src="http://placehold.it/900x350" alt="Third slide">
      </div>
    </div>
    <a class="carousel-control-prev" href="#carouselExampleIndicators" role="button" data-slide="prev">
      <span class="carousel-control-prev-icon" aria-hidden="true"></span>
      <span class="sr-only">Previous</span>
    </a>
    <a class="carousel-control-next" href="#carouselExampleIndicators" role="button" data-slide="next">
      <span class="carousel-control-next-icon" aria-hidden="true"></span>
      <span class="sr-only">Next</span>
    </a>
  </div>
{% endif %}

<div class="row">
  {% for prod in product_list %}
    <div class="col-lg-4 col-md-6 mb-4">
      <div class="card h-100">
        <a href="{% url 'tienda:producto' prod.id %}">
          {% if prod.imagen %}
            <img class="card-img-top" src="{{ prod.imagen.url }}" alt="{{ prod.nombre }}" style="height: 200px; object-fit: cover;">
          {% else %}
            <img class="card-img-top" src="http://placehold.it/700x400" alt="Sin Imagen">
          {% endif %}
        </a>
        <div class="card-body">
          <h4 class="card-title">
            <a href="{% url 'tienda:producto' prod.id %}">{{ prod.nombre }}</a>
          </h4>
          <h5>S/ {{ prod.precio }}</h5>
          <p class="card-text">Categoría: {{ prod.categoria.nombre }}</p>
        </div>
        <div class="card-footer">
          <small class="text-muted">★ ★ ★ ★ ☆</small>
        </div>
      </div>
    </div>
  {% empty %}
    <div class="col-12">
      <p>No hay productos registrados en esta categoría.</p>
    </div>
  {% endfor %}
</div>
{% endblock %}
```

Figura 35. Actualización de producto.html para mostrar la imagen del producto

producto.html

tienda > templates > producto.html > ...

```
{% extends "layout.html" %}

{% block content %}
<div class="card mt-4">
  {% if producto.imagen %}
    <img class="card-img-top img-fluid" src="{{ producto.imagen.url }}" alt="{{ producto.nombre }}" style="max-height: 400px; object-fit: contain;"
  {% else %}
    <img class="card-img-top img-fluid" src="http://placehold.it/900x400" alt="">
  {% endif %}
  <div class="card-body">
    <h3 class="card-title">{{ producto.nombre }}</h3>
    <h4>S/ {{ producto.precio }}</h4>
    <p class="card-text">Categoría: {{ producto.categoria.nombre }}</p>
    <p class="card-text">Stock disponible: {{ producto.stock }} unidades</p>
  </div>
</div>
{% endblock %}
```

Figura 36. Una foto por cada categoría

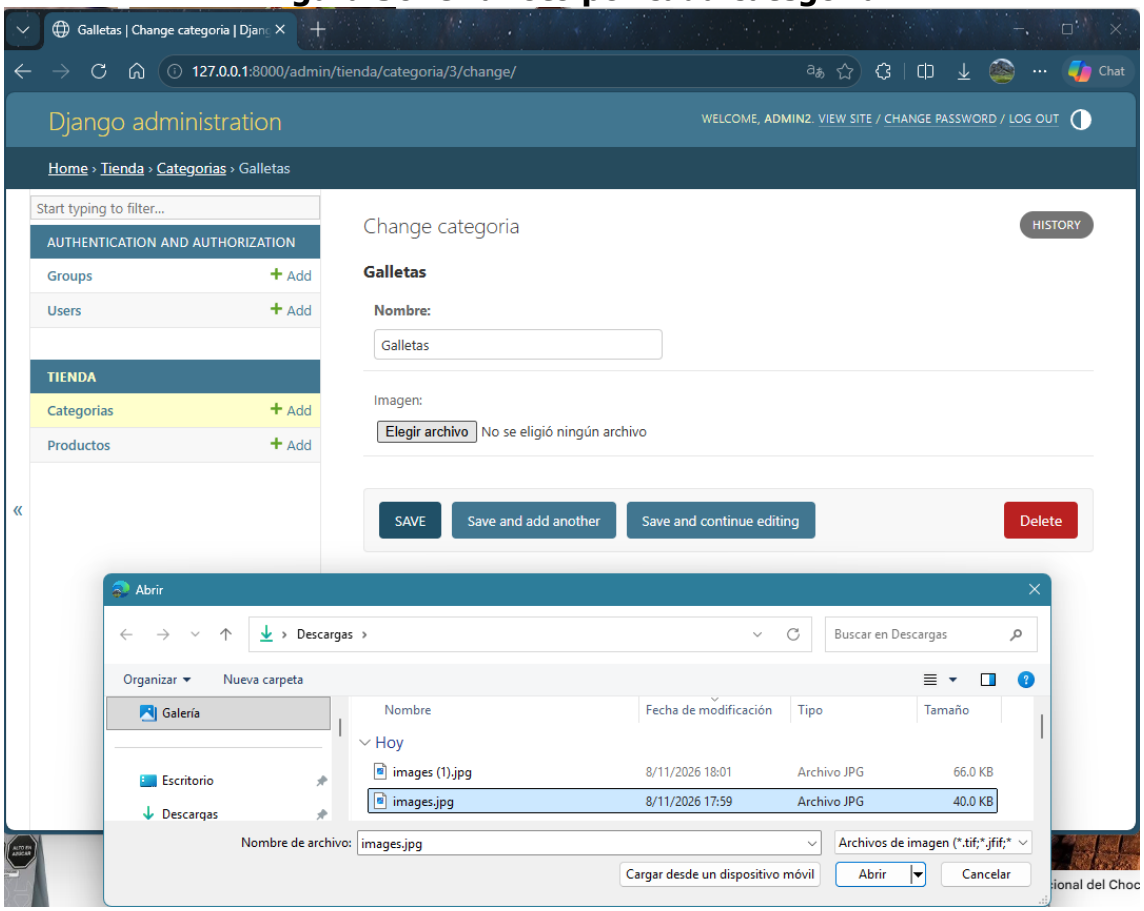


Figura 37. Una foto por cada producto

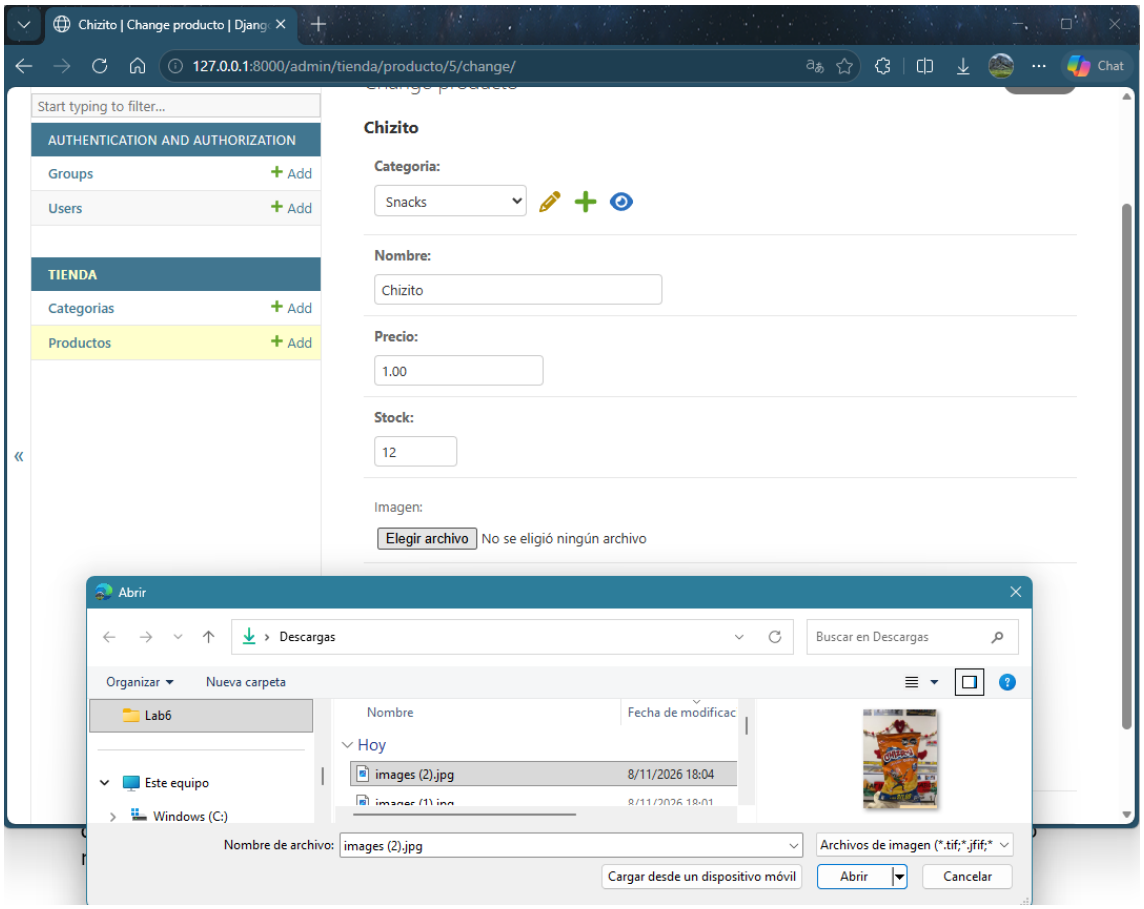


Figura 38. Menú lateral y fotos de productos en Inicio

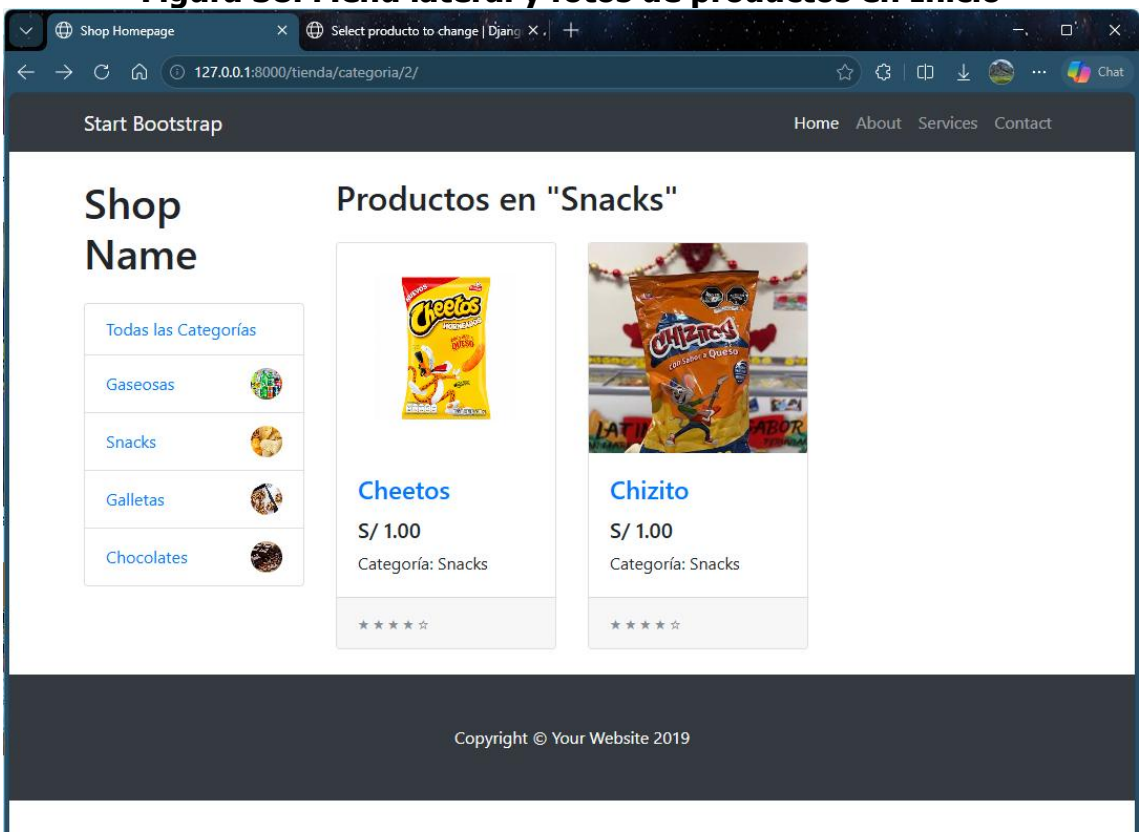
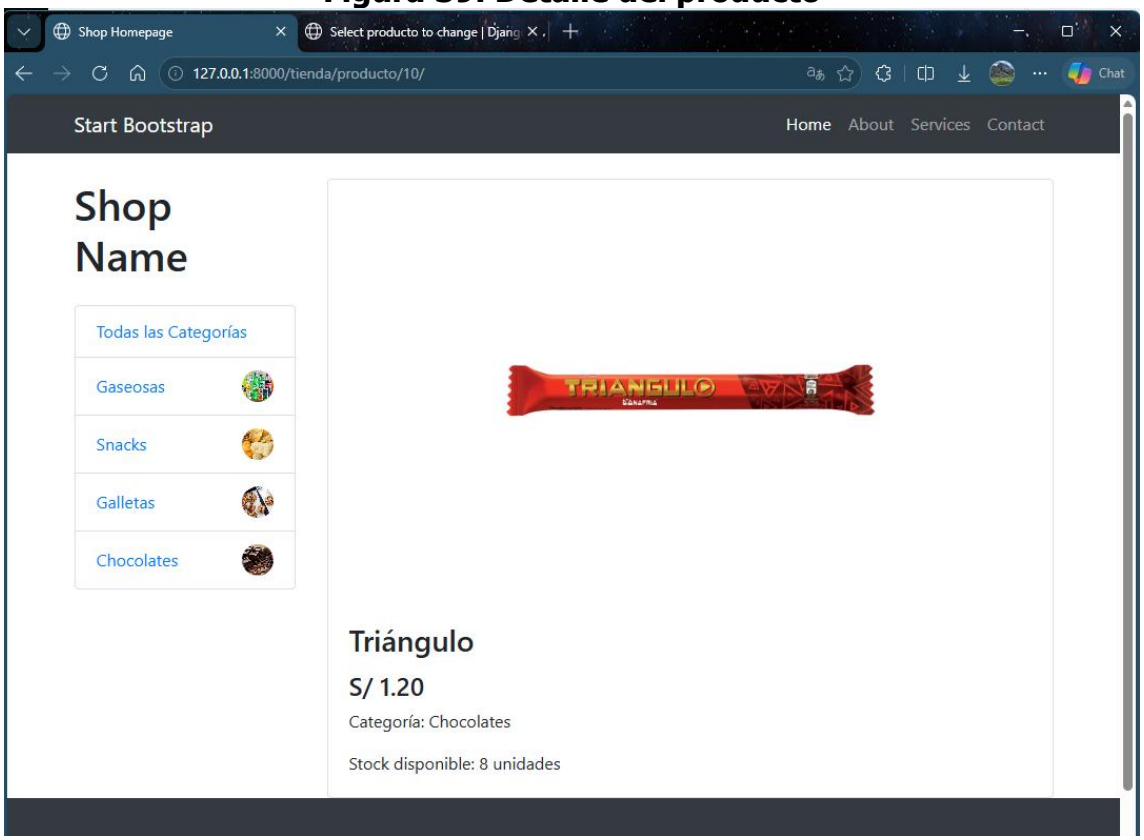


Figura 39. Detalle del producto



CONCLUSIONES:

1. La arquitectura MVT de Django y su motor de plantillas (DTL) simplifican enormemente el desarrollo web al separar la lógica de negocio en las vistas de la capa de presentación HTML, lo que permite un código más limpio y fácil de mantener.

2. La implementación de una plantilla base (layout.html) con etiquetas {% extends %} y {% block %} redujo drásticamente la duplicación de código HTML y hojas de estilo, garantizando una estética uniforme en toda la tienda.

3. El uso de un context processor personalizado para cargar las categorías fue un acierto clave, ya que permitió alimentar el menú lateral de forma automática en todas las páginas sin repetir consultas a la base de datos en cada vista individual.

4. Incorporar el modelo de imágenes mediante ImageField demostró lo práctico que es integrar soporte multimedia en Django, conectando fácilmente la subida de archivos desde el panel de administración con su despliegue visual en el frontend.

5. La configuración de rutas parametrizadas en urls.py facilitó que los usuarios exploren los productos por categoría de forma fluida, demostrando cómo las vistas filtradas mejoran la experiencia de navegación en una plataforma de comercio electrónico.

Criterios de evaluación

Tecsup	***Desarrollo de Aplicaciones Web***				
	Rúbrica holística				
Elemento de la capacidad					
Curso				**Periodo**	
Actividad				**Semestre**	
Nombre del alumno				**Semana**	
Docente		**Fecha**		**Sección**	

Criterios a evaluar	**Excelente**	**Bueno**	**Requiere mejora**	**No aceptable**	**Puntaje logrado**
Implementación de template base y herencia correcta					
Uso correcto de tags y variables de template					
Integración de archivos estáticos (CSS, imágenes)					
Renderización correcta de páginas con contexto dinámico					
Tarea adicional: template con filtros y lógica condicional					
Total					

Acciones a cumplir	***Menos***
● Puntualidad y dedicación	1
● Cumplimiento de tiempos establecidos	1
● Conclusiones: ortografía y redacción	1
Puntaje total	

Comentarios respecto del desempeño del alumno	

	Descripción
Excelente	Demuestra un completo entendimiento del problema o realiza la actividad cumpliendo todos los requerimientos especificados.
Bueno	Demuestra un considerable entendimiento del problema o realiza la actividad cumpliendo con la mayoría de los requerimientos especificados.
Requiere mejora	Demuestra un bajo entendimiento del problema o realiza la actividad con pocos de los requerimientos especificados.
No aceptable	No demuestra entendimiento del problema o actividad.